Quartet: A Digital Compute-in-Memory Versatile AI Accelerator With Heterogeneous Tensor Engines and Off-Chip-Less Dataflow

Yikan Qiu, Guoxiang Li, Meng Wu, *Graduate Student Member, IEEE*, Yifan Jia, Le Ye, *Senior Member, IEEE*, and Yufei Ma, *Senior Member, IEEE*

***Abstract*—Although most AI core operations can be formulated as matrix multiplications (MMs), their characteristics are quite different. In some cases, one input may be a constant weight matrix while the other is a dynamic feature matrix, i.e., FWMM, or both inputs may be dynamic features as FFMM. Furthermore, the data involved can also be sparse, leading to variations like SpFWMM and SpFFMM. To address this challenge, this paper investigates a versatile accelerator architecture for AI algorithms based on heterogeneous tensor engines. For MM operators with varying characteristics, this paper proposes four-quadrant heterogeneous tensor engines to handle FWMM, SpFWMM, FFMM, and SpFFMM, respectively. These four tensor engines are comprised of SRAM-based single address digital compute-in-memory (CIM) array, SRAM-based multi-address digital CIM array, systolic array, and multi-SIMD array, respectively. In addition, to improve the AI execution efficiency, this paper proposes a dual-level multi-issue mechanism to achieve inter-operator and inter-block parallelization along with an off-chip-less dataflow enabled by on-chip unified memory pool. Thanks to the integration of aforementioned innovations, this paper develops a versatile AI acceleration chip Quartet, which achieves exceptional energy efficiency. Specifically, for graph convolutional network on PubMed, it demonstrates a 19.56× and 3.47× improvement in energy efficiency compared to similar works, ReDCIM and TensorCIM, respectively.**

***Index Terms*—Domain-specific accelerator, compute-in-memory, heterogeneous architecture, tensor engine, graph neural networks, convolutional neural networks, transformer.**

I. Introduction

ALTHOUGH the core operations in various AI models can be formulated as matrix multiplication (MM), these MMs exhibit distinct characteristics as shown in Fig. 1. The Q-K-V generation in transformer [1], combination phase in graph convolutional network (GCN) [2], and convolution layers in convolutional neural networks (CNNs) [3], [4], [5], [6], [7] involve static MM with constant weights, which can be leveraged by compute-in-memory (CIM) [8], [9], [10] to eliminate costly weight data movements. However, the dominant MM of attention in transformer, aggregation in graph neural network (GNN), and graph construction in vision GNN (ViG) [11] is dynamic that neither input is constant, degrading the benefits brought by CIM. In addition, the varying sparsity of MM in different operators typically demands different zero-skipping granularity, leading to different hardware overheads.

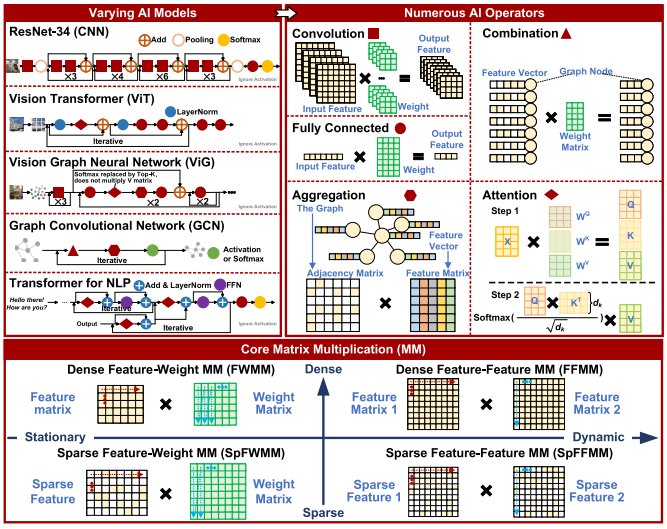


Fig. 1. The core operations in AI models can be mainly categorized as feature-weight MM (FWMM), sparse feature-weight MM (SpFWMM), feature-feature MM (FFMM) and sparse feature-feature MM (SpFFMM).

Therefore, a domain specific AI accelerator faces three main challenges: 1) the customized design scheme for numerous and every-changing AI operators or models leads to excessive and divergent hardware modules, limiting flexibility and overall utilization [12], [13], [14], [15], [16], [17], [18], [19], [20]; 2) the unified computing array based on CIM cannot efficiently and suitably process MM with varying sparsity, scale, and data formats [21], [22], [23], [24], [25]; 3) the massive data movements between adjacent operators cause frequent and intensive off-chip memory accesses, resulting in high latency and energy consumption.

To address these challenges, we propose a versatile AI accelerator Quartet based on digital CIM (DCIM) in

Received 10 March 2025; revised 2 June 2025 and 17 July 2025; accepted 6 August 2025. Date of publication 19 August 2025; date of current version 12 January 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB4404603; and in part by the National Natural Science Foundation of China under Grant 92164301, Grant 62204003, Grant 62225401, and Grant 61927901. This article was recommended by Associate Editor H. Cai. *(Yikan Qiu and Guoxiang Li are co-first authors.) (Corresponding author: Yufei Ma.)*

Yikan Qiu, Guoxiang Li, Yifan Jia, Le Ye, and Yufei Ma are with the Institute for Artificial Intelligence and the School of Integrated Circuits, Peking University, Beijing 100871, China (e-mail: yufei.ma@pku.edu.cn).

Meng Wu is with the School of Integrated Circuits, Peking University, Beijing 100871, China.

Digital Object Identifier 10.1109/TCSI.2025.3598287

1549-8328 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

commercial 22nm to accelerate a diverse range of AI models, including transformers, GCNs, ViG, CNNs, and beyond. The major contributions of this paper are as follows:

- A two-tier multiple-issue overall architecture is proposed, featuring four heterogeneous tensor engines (TEs) that can be dynamically scheduled to execute various AI operators in parallel or in pipeline.
- Four TEs based on multiple-address DCIM, single address DCIM, systolic array, and single instruction multiple data (SIMD) micro-architecture are tailored for various static or dynamic MMs with sparse or dense data.
- The local buffers in each TE is interconnected through network on chip (NoC) to construct a unified memory pool, enabling configurable communication and data sharing among TEs to minimize off-chip memory accesses.
- Thanks to the integration of aforementioned innovations, Quartet achieves exceptional energy efficiency. Specifically, for GCN on PubMed, it demonstrates 19.56× and 3.47× improvements in energy efficiency compared to similar works, ReDCIM [26], [27] and TensorCIM [28], respectively.

The rest of this article is organized as follows. Section II presents the overall architecture of Quartet. Section III introduces the feature-weight MM tensor engine and the design analysis of DCIM. Section IV describes sparse feature-weight MM tensor engine. Section V presents feature-feature MM tensor engine and Section VI displays sparse feature-feature MM tensor engine. Section VII introduces the design of sort engine and nonlinear engine. Section VIII shows the implementation and measurement results of Quartet to validate the efficiency. Section IX makes conclusions.

II. Overall Architecture of Quartet

A. Overall Architecture

The proposed versatile AI accelerator Quartet as shown in Fig. 2 is composed of instruction memory, global controller, NoC, and four TEs. The global controller fetches instruction from instruction memory, sends control signal to, and receives status signal from four TEs and NoC. The multiple-address DCIM, which enhances the flexibility to fetch corresponding weights with sparse inputs, is proposed for MM with sparse (Sp) feature-weight (FW) inputs, i.e., Sparse Feature Weight Matrix Multiplication (SpFWMM), while the single-address DCIM is employed for MM with dense FW inputs, i.e., Feature Weight Matrix Multiplication (FWMM). The TE based on systolic array is designed for MM with dynamic feature-feature (FF) inputs, i.e., Feature Feature Matrix Multiplication (FFMM), and the SIMD-based TE is proposed for MM with sparse FF inputs, i.e., Sparse Feature Feature Matrix Multiplication (SpFFMM). Quertet uses DCIM to handle FWMM and digital PE for FFMM. This is because the advantage of DCIM lies in its ability to reduce the overhead of transferring data between on-chip storage and computation, as both are integrated within the DCIM. When processing FWMM, the weights typically have more reuse opportunities and do not require frequent refreshing, allowing for effective utilization of DCIM's benefits. However, when processing FFMM, the two

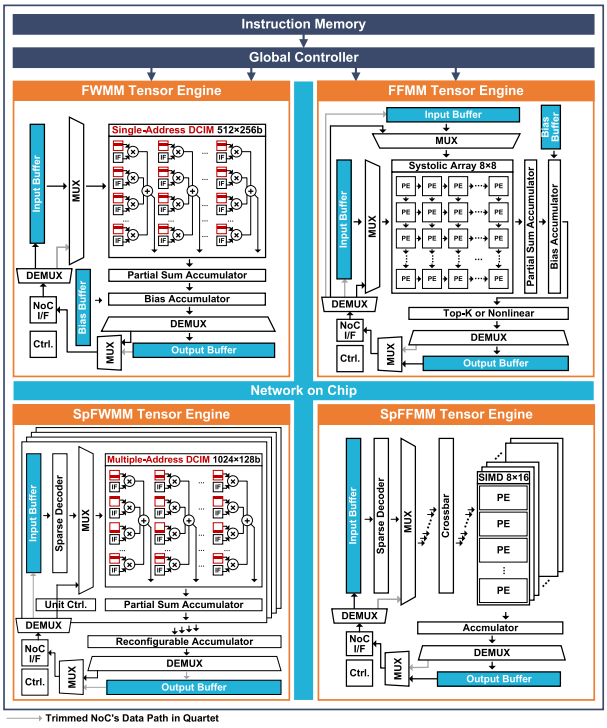


Fig. 2. The overall architecture of Quartet with four heterogeneous TEs.

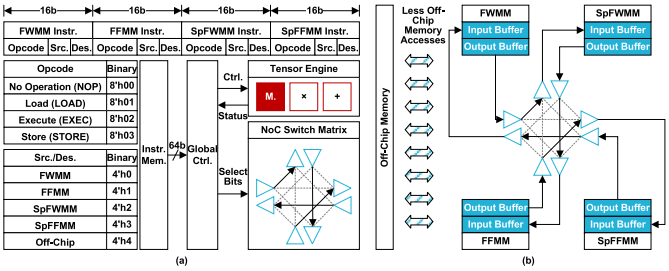


Fig. 3. (a) VLIW-based instruction set architecture for multiple-issue scheme. (b) Unified memory pool by interconnecting distributed buffers through NoC.

matrices involved in multiplication often need to be refreshed frequently, necessitating interactions with off-chip DDR. The overhead of on-chip to off-chip data transfer is much greater than that of on-chip data transfer, which diminishes the advantages of DCIM. The digital PE array, on the other hand, offers greater flexibility and supports various data flows, which is why digital PE is used to process FFMM.

Each TE is equipped with configuration registers that can be controlled by the host, NoC interfaces for communication with other TEs or off-chip memory, unified local memory shared among TEs, and computing elements (CEs) for specific MM. The NoC is responsible for transferring data among TEs, facilitating the efficient execution of complex AI workloads.

To fully harness the capabilities of the four heterogeneous tensor computation engines, we have designed a corresponding Very Long Instruction Word (VLIW) architecture for Quartet, as shown in Fig. 3(a). Each VLIW comprises instructions tailored for these four heterogeneous TEs. An instruction is structured with an opcode and two operands. Our design incorporates four types of opcodes: No Operation (NOP),

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

Load (LD), Execute (EX), and Store (ST). Additionally, there are five types of operands, which correspond to the addresses of each TE and the address of off-chip memory. Upon receiving an external start signal, the global controller initiates instruction fetching. These fetched instructions are then decoded and translated into control signals for the TEs and NoC. This design enables the four tensor engines to operate simultaneously.

B. Two-Tier Multiple-Issue Scheme

The multiple-issue architecture in Quartet can be categorized into two tiers. The first tier lies between operators, enabling the parallelization of branches within a neural network's directed acyclic graph. The second tier is within operators. Typically, the size of neural network data varies, necessitating its division into tiles to suit hardware computation. If these operators are further segmented into smaller tiles without data dependency, their computation can be parallelized.

In Quartet, various operators, such as convolution operators in CNNs and aggregation operators in GCNs, can be computed within the FWMM tensor engine and the SpFFMM tensor engine, respectively, by leveraging Quartet's multi-issue instructions as depicted in Fig. 3. Additionally, different blocks, such as combination operations in GCNs, can utilize the SRAM-based multi-address DCIM in individual mode within the SpFWMM tensor engine, thereby achieving inter-block parallelism for operators of the same type.

C. Unified Memory Pool

Off-chip data accesses pose a significant constraint on the energy efficiency of AI accelerators. To address this challenge, Quartet adopts a distributed shared memory (DSM) architecture, as illustrated in Fig. 3(b). This architecture offers the advantage of achieving higher memory bandwidth and lower local memory access latency compared to the symmetric shared memory (SSM) architecture. The key lies in that the local buffers in each tensor engine can be interconnected to construct a unified memory pool, allowing all local buffers to be shared among TEs. In this architecture, each tensor engine communicates data with other engines through a shared address space. The Quartet defines the shared address space. The high 2 bits of the 32-bit address represent the tensor engine, and the low 30 bits are used to represent the address within the tensor engine. This architecture offers two key advantages. Firstly, it supports computation pipelining, enabling the output of one TE to be fed into another. Secondly, DCIM-based TEs can flexibly serve as computation engines when computation is required and as memory when computation is not needed. In the latter case, such as QKV computation in Transformer and aggregation of GNN, the non-CIM TEs can utilize the DCIM-based TEs as shared memory, further reducing the necessity for off-chip data communication for non-CIM TEs. In this architecture, each TE performs data communication between the engines through a shared address space. Quartet defines a shared address space where the most significant 2 bits of the 32-bit address indicate the number of a TE, while the lower 30 bits are used to indicate the address within the TE.

To support this distributed shared memory architecture, we have designed the interface between NoC and TEs. Specifically, the NoC is structured as a crossbar, allowing each TE to communicate directly with the other three TEs for data exchange. It is flexible enough to allow the inputs to be either stored in input buffer or directly sent to MM computing array, and similarly for the outputs. The on-chip network interface of the TEs comprises a demultiplexer, a multiplexer, and a data translator. The demultiplexer is responsible for distributing control, address, and data signals from the interconnection network to the memory or the computational array, depending on the address ranges. The multiplexer selects the output from either the computational array or the memory to be transmitted to the NoC. Lastly, the data translator converts data received from other TEs in different formats into the format used by the current TE.

D. Vector-Element Co-Optimization Technique

When processing SPMM from a vector perspective, there are fewer non-zero elements within the vector, resulting in low utilization of the computing array. The vectorization perspective means that during parallel computation, the data processed in parallel is viewed as a vector. Conversely, if processed from an element perspective, memory access conflicts occur due to multiple elements targeting different addresses in the same memory bank. This forces parallel memory accesses to be serialized, also leading to low utilization.

This paper proposes a vector-element co-optimization technique that significantly improves computation modules' utilization compared to either vector-only or element-only processing approaches. The principle is illustrated in Fig. 4. Starting from a vector perspective, the technique first applies strip mining to the sparse matrix (similar to dense matrix processing). It then swaps data from the same positions across different input vectors, pushing zeros to the right, which can potentially result in input vectors that are entirely zeros, thereby increasing intra-vector non-zero density and reducing the total number of non-zero vectors. The implementation is detailed in Algorithm 1. Notably, dimension i in this algorithm has no cross-iteration dependencies, enabling parallelization through loop unrolling.

From an element perspective, vectors are first combined according to the pattern of consecutive non-zero elements. Then, based on the memory access conditions caused by the indices of non-zero elements within the vectors, the non-zero elements are extrapolated to other vectors, thereby achieving an increase in the number of non-zero elements within vectors and a reduction in the number of vectors. Correspondingly, its implementation is shown in Algorithm 2. Similar to Algorithm 1, the dimension i in this algorithm also has no cross-iteration dependencies, so it can be parallelized by loop unrolling.

The choice of the algorithm is determined by the storage format of the sparse matrix X. If X is stored densely, Algorithm 1 is used. If X is stored in a sparse form, such as COO, CSR and CSC, then Algorithm 2 is adopted. The

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Vector-Element Co-Optimization Technique Algorithm From a Vector Perspective

Require: Sparse matrix X, vector length L
Ensure: Group row index array *SROW*, group column index matrix *SCOL*, group value matrix *SVAL*
Initialize *SROW* as a zero array
Initialize *SCOL* and *SVAL* as zero matrices
Initialize *Ptr* to 0
N_{Row} = number of rows in sparse matrix X
N_{Col} = number of columns in sparse matrix X
$N_{Max\ Vector} = \lceil N_{Col}/L \rceil$
for $i = 1 \rightarrow N_{Row}$ **do**
  for $j = 1 \rightarrow N_{Max\ Vector}$ **do**
    for $k = 1 \rightarrow L$ **do**
      if $X[i, j \cdot L + k] == 0$ **then**
        for $l = j + 1 \rightarrow N_{Max\ Vector}$ **do**
          if $X[i, l \cdot L + k]! = 0$ **then**
            Swap $X[i, l \cdot L + k]$ and $X[i, j \cdot L + k]$
            break
          end if
        end for
      end if
    end for
  end for
  for $j = 1 \rightarrow N_{Max\ Vector}$ **do**
    if non-zero elements exist in $X[i, j \cdot L : (j + 1) \cdot L]$ **then**
      $SROW[Ptr] = i$
      Assign the corresponding column indices of $X[i, j \cdot L : (j + 1) \cdot L]$ to $SCOL[Ptr, :]$
      Assign the corresponding values of $X[i, j \cdot L : (j + 1) \cdot L]$ to $SVAL[Ptr, :]$
      $Ptr = Ptr + 1$
    end if
  end for
end for

Algorithm 2 Vector-Element Co-Optimization Technique Algorithm From Element Perspective

Require: Sparse matrix X in CSR format, vector length L
Ensure: Group row index array $SROW$, group column index matrix $SCOL$, group value matrix $SVAL$
Initialize $SROW$ as an array of zeros
Initialize $SCOL$ and $SVAL$ as zero matrices
Initialize pointer *Ptr* as zero
N_{Row} = number of rows in sparse matrix X
N_{Col} = number of columns in sparse matrix X
$N_{Max\ Vector} = \lceil N_{Col}/L \rceil$
for $i = 1 \rightarrow N_{Row}$ **do**
  Initialize group index array *GIND* as zeros of length L
  for $p = X_{\text{CSR}}.\text{indptr}[i] \rightarrow X_{\text{CSR}}.\text{indptr}[i + 1]$ **do**
    $c = X_{\text{CSR}}.\text{indices}[p]$ {Column index of current element}
    $g = c \mod L$ {Group within the vector}
    $k = GIND[g]$ {Position within the group}
    $X[i, k \cdot L + g] = X_{\text{CSR}}.\text{data}[p]$ {Reorder element into new position}
    $GIND[g] = GIND[g] + 1$
  end for
  for $j = 1 \rightarrow N_{Max\ Vector}$ **do**
    if non-zero elements exist in $X[i, j \cdot L : (j + 1) \cdot L]$ **then**
      $SROW[Ptr] = i$
      Assign the corresponding column indices of $X[i, j \cdot L : (j + 1) \cdot L]$ to $SCOL[Ptr, :]$
      Assign the corresponding values of $X[i, j \cdot L : (j + 1) \cdot L]$ to $SVAL[Ptr, :]$
      $Ptr = Ptr + 1$
    end if
  end for
end for

outermost loops of both algorithms have no cross-iteration dependencies, so both can be parallelized by loop unrolling.

III. FWMM Tensor Engine

The feature-weight MM is exemplified by convolution operator in CNNs, which is notable for its intensive computational demands and ample opportunities for data reuse. In prevalent CNN models today, the convolution operation exhibits a significantly higher number of MAC (Multiply-and-Accumulate) operations compared to activations and weights. For instance, the Conv1 layer of ResNet-50 demonstrates a stark contrast, with 147 times more MACs than activations and 12,461 times more than weights. This disparity underscores the substantial potential for data reuse. Consequently, a compute-in-memory architecture is employed within the tensor engine to capitalize on these data reuse opportunities and minimize data movement.

In traditional DCIM architecture, one memory cell typically corresponds to one processing element. Conversely, we employ a subarray structure where multiple memory cells are associated with a single processing element. The DCIM array also functions as a buffer for the tensor engine. The DCIM buffer size directly affects the frequency of off-chip memory accesses. Specifically, a larger on-chip buffer leads to fewer off-chip memory accesses. To further minimize off-chip memory accesses and enhance memory size per throughput, the adoption of the subarray structure is crucial.

Regarding to the algorithm mapping scheme, weights are rearranged, sliced, padded, and stored in the DCIM, while feature data are similarly rearranged, sliced, padded, and sent to the DCIM. As depicted in Fig. 5, the feature data in the FWMM exhibit a continuous and dense characteristic. The weights are stored in the DCIM in an interleaved manner, ensuring that the weight corresponding to each feature is stored at the same address in the subarray for every computation. Therefore, a single-address DCIM suffices for the FWMM computation.

The following considerations are made for selecting the subarray depth for a single-address DCIM. The SRAM-based DCIM serves as a buffer for the tensor engine, and its size should be maximized to minimize off-chip accesses of weights. However, a larger subarray depth will result in slower data

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

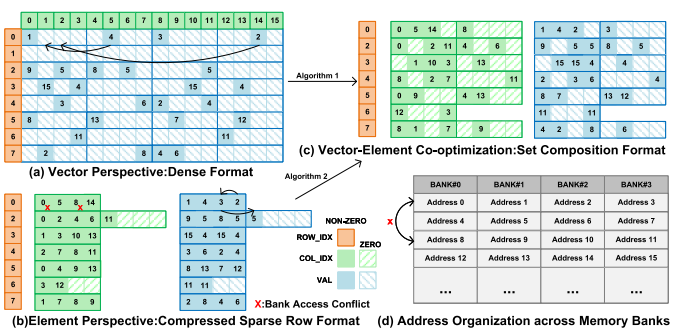


Fig. 4. (a) Dense Format leads to huge waste; (b) CSR (Compressed Sparse Row) Format causes bank conflict; (c) Through transformation of Algorithm 1 or 2, our proposed SC (Set-Composition) Format can be obtained. SC Format has the characteristics of intra-bank randomness and inter-bank continuity; (d) Address organization: Each bank can only access one address at a time, and accessing two addresses simultaneously will cause conflict.

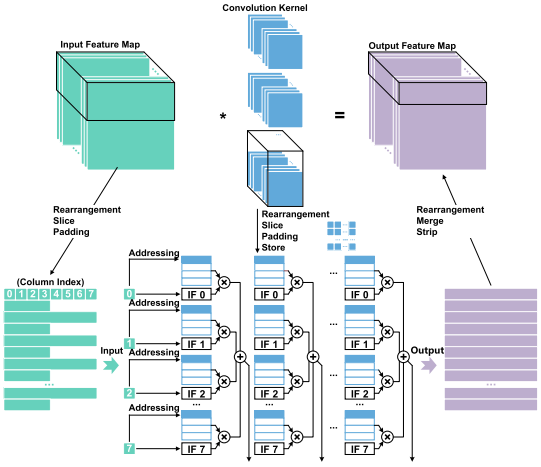


Fig. 5. The mapping scheme of convolution operation to FWMM tensor engine with single-address DCIM.

read speeds, thereby limiting the computation speed of DCIM by the subarray read speed. Specifically, for a subarray with a depth of 2^m and a width of 2^n, the latency of the read operation [29] is

$$D_{read} = (2(n+m) + 4\log_4((m+2)/3) + m + 2^{m+1}/3)\tau, \quad (1)$$

where τ is the delay of fan-out-1 inverter without parasitic capacitance. Meanwhile, the computing delay of subarray is

$$D_{compute} = D_{multiply} + D_{add}, \quad (2)$$

where $D_{multiply}$ is the delay of multiplication operation and D_{add} is the delay of addition operation. Since the multiplication operation is performed by a NOR gate, then

$$D_{multiply} = D_{NOR} = 9\tau. \quad (3)$$

The addition operation is performed by an adder tree composed of multiple adders, and for a k-input DCIM array,

$$D_{add} = ((9 + \log_2 k)\cdot\log_2 k)/2 \cdot D_{FA} \quad (4)$$

$$= ((9 + \log_2 k)\cdot\log_2 k)/2 \cdot 10\tau. \quad (5)$$

where the delay of the first-stage adder is

$$D_{add0} = 5 \times 10\tau = 50\tau. \quad (6)$$

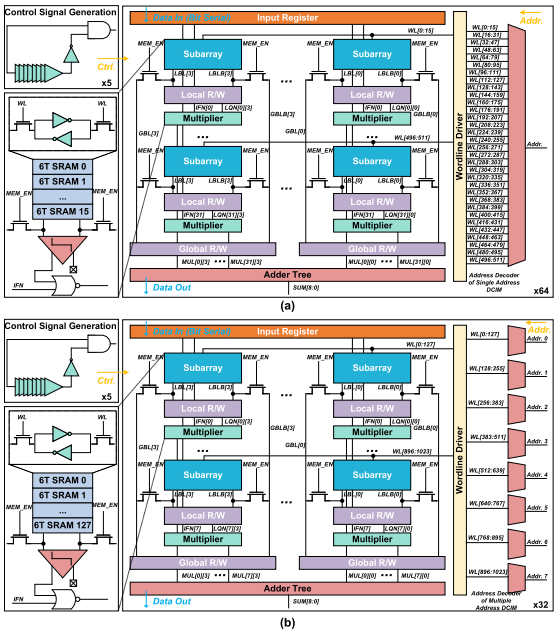


Fig. 6. (a) The circuit architecture of SRAM-based digital CIM with address decoder for single-address DCIM. (b) The circuit architecture of multiple-address DCIM.

Thus, the delay of the multiplier and the first-stage adder is

$$D_{compute0} = 59\tau. \quad (7)$$

D_{read} should be less than or equal to $D_{compute0}$ so that the read operation does not limit the speed of the DCIM. When $m > 5$, the latency of the subarray read operation surpasses the computational latency of both the multiplication and the first-stage adder. Consequently, a memory-computation ratio of 16 and 32 represents a relatively optimal choice. This way, a computation unit can utilize data from 16 or 32 storage units, which is more conducive to weight reuse and can reduce the number of refreshes in the DCIM.

Based on the analysis, we have designed an single-address DCIM array for MVM, as shown in Fig. 6(a). This DCIM array boasts a memory size of 512×256 bits and a memory-computation ratio of 16. Structurally, it comprises two primary components: the control circuit and the DCIM array circuit. The control circuit itself is divided into a control signal generator and an address decoder. The control signal generator is responsible for generating the pulse signals for the DCIM array. Meanwhile, the address decoding circuit translates address signals into the word line signals. This is accomplished using a multiplexer that allows all subarrays to share an address, alongside a set of word lines. The array circuit itself is uniformly organized into 64 columns. Each column features one set of registers, one adder tree, and four subcolumns. These subcolumns are each composed of a global read/write circuit, 32 subarrays, 32 local read/write circuits, and 32 multipliers.

IV. SpFWMM Tensor Engine

The sparse feature-weight MM (SpFWMM), represented by the combination operator in GCNs, shares the requirement for static weight data in computations with the standard FWMM

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

operation. However, the key difference lies in the discontinuous addressing of weights. This is because, for the combination operations in GCNs, the feature matrix is expressed as a sparse matrix. To enhance utilization efficiency, this sparse matrix necessitates the application of vector-element co-optimization techniques. Consider the first row in Fig. 4(a), suppose this represents the first row of the input matrix to DCIM, and the DCIM input vector length is 4, meaning that the DCIM has 4 rows of subarrays. Without optimization, completing the computation of the first row requires four calculations. During these four computations, the subarray addresses in DCIM are 0, 1, 2, and 3, corresponding to the selected weight data. By optimizing, we swap data from the same positions across different input vectors, pushing zeros to the right, which can result in input vectors that are entirely zero. These input vectors can be skipped. As shown in the first row of Fig. 4(c), it only contains two input vectors. However, when DCIM computes, input data 1 needs to be multiplied by the weight at subarray address 0, input data 4 by the weight at subarray address 1, and input data 2 by the weight at subarray address 3. A single-address DCIM cannot fulfil this requirement. In all, the discontinuity in indices within the vector results in different addresses being accessed for each subarray, making a single-address DCIM insufficient for such operations. Therefore, we propose the design of a multiple-address DCIM to support these operations effectively.

Like the FWMM tensor engine, the SpFWMM tensor engine also demands a subarray structure to increase the DCIM memory size as a buffer. The depth of subarray in the multiple address DCIM is contemplated as follows. On one hand, increasing the depth of the subarray results in a slower data read speed. A subarray with a depth of 16 can ensure that the speed of the DCIM is not constrained by data read operations. On the other hand, for SpFWMM, employing vector-element co-optimization techniques, a deeper subarray is advantageous for diminishing the number of vectors, enhancing the utilization of DCIM, and consequently reducing computational latency. For a SpFWMM, its computation time on a multiple address DCIM, denoted as D_{cim}, is given by:

$$D_{cim} = T_{cycle} \cdot N_{vec} = max(D_{read}, D_{compute0}) \cdot N_{vec}, \quad (8)$$

where T_{cycle} is the clock cycle period, N_{Vec} is the number of vectors after employing the vector-element co-optimization technique, D_{read} is the latency of reading data from the subarray, and $D_{compute0}$ is the computational latency of multiplier and the first-stage adder.

The variation of D_{cim} with subarray depth is illustrated in Fig. 8(a) for an input vector length of 8. It is evident that as the subarray depth increases to 256 and 512, the computation time of the DCIM experiences a substantial increase. However, at subarray depths of 16, 32, 64, and 128, the differences in computation time are relatively minor. On the other hand, a larger subarray depth increases the size of the DCIM, ultimately decreasing the number of output buffer accesses. The variation in output buffer accesses with subarray depth is depicted in Fig. 8(b). Taking into account both computation speed and the number of output buffer accesses, a subarray depth of 128 emerges as a relatively reasonable choice.

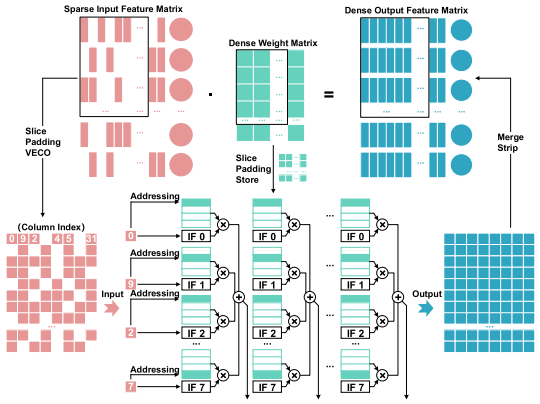


Fig. 7. The mapping scheme of combination operation to SpFWMM tensor engine with multiple-address DCIM.

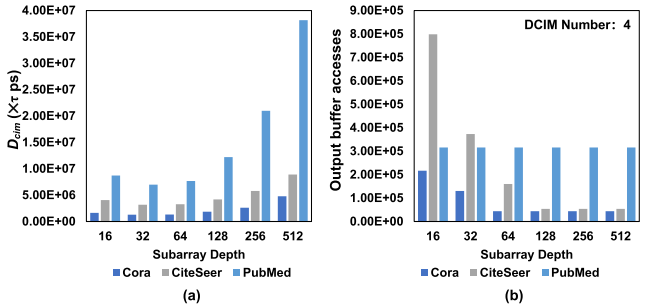


Fig. 8. The variation of D_{cim} and output buffer accesses with subarray depth.

Based on the aforementioned analysis, we have designed an SRAM-based multiple-address DCIM array tailored for sparse MVM as shown in Fig. 7. This array possesses a memory size of 1024×128 bits and a memory-computation ratio of 128. The primary distinction from the single address DCIM array lies in the utilization of an independent address decoder, as shown in Fig. 6(b). Compared to FWMM, the architecture of SpFWMM TE incorporates configurable adder trees to facilitate MVM with varying lengths. Leveraging it, the SpFWMM TE can operate in three distinct modes: independent mode, 2-way collaborative mode, and 4-way collaborative mode. These modes are designed to process MVM for lengths less than 1024, less than 2048, and less than 4096, respectively. To facilitate seamless switching between operating modes, the global controller is equipped with a configuration register denoted as *sw_opmode*. When *sw_opmode* is set to 0, the SpFWMM TE operates in independent mode, where each multiple address DCIM array performs computations independently. When *sw_opmode* is set to 1, the SpFWMM TE transitions to the 2-way collaboration mode, where DCIM 0 and 1 work together to complete computation tasks, while DCIM 2 and 3 also collaborate on their respective tasks. Lastly, when *sw_opmode* is set to 2 or 3, the SpFWMM TE operates in the 4-way collaboration mode, where DCIM 0, 1, 2, and 3 work collectively to accomplish the computation task.

V. FFMM Tensor Engine

The feature-feature matrix multiplication (FFMM) is exemplified by the attention mechanism in Transformer and the graph construction operator in Vision GNN. It is characterized

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

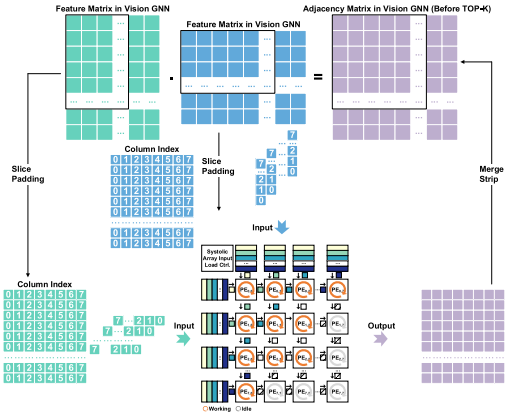


Fig. 9. Mapping of graph construction to FFMM tensor engine.

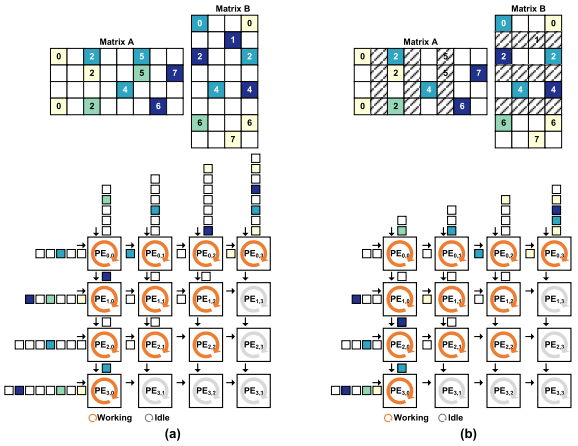


Fig. 10. Systolic array processes sparse MM: (a) treat sparse matrix as dense matrix; (b) use row-wise/column-wise sparsity-aware computation reduction.

by the fact that the input data are all generated by the preceding stage, devoid of the static characteristics inherent in FWMM and SpFWMM. If FFMM is also processed by the DCIM, it would result in overly frequent update operations of the DCIM, significantly increasing computational delay and energy consumption. Therefore, an output-stationary systolic array is employed to support FFMM. This systolic array enhances data reuse by augmenting the data paths between processing elements, thereby reducing the dependence of processing elements on memory, as illustrated in Fig. 9.

The FFMM tensor engine is constructed with an output stationary systolic array as its core computational component. The systolic array and its peripheral circuits consist of two systolic data setup units and one systolic array unit. The systolic data setup units are responsible for converting parallel data into systolic data, while the systolic array unit is responsible for performing the matrix multiplication operation. In addition to the systolic array, the FFMM tensor engine also comprises input buffers, input load controllers equipped with systolic data setup in the front stage, as well as output load controllers, bias buffers, bias accumulators (BAs), quantizers (RQs), and output buffers in the back stage.

The data within the FFMM is organized in a matrix-like format within the input buffer. Unlike the FWMM and SpFWMM, which utilize the weight-stationary mode, the FFMM stores the output data in each PE. Additional output load controllers need to be integrated into the backend circuit to retrieve the output data stored in the systolic array.

To enhance the flexibility of the systolic array for computing various input feature matrices and to introduce configurability to the FFMM tensor engine, the input load controller is equipped with seven configuration registers. Specifically, $sw_addr_base_ma$ and $sw_addr_limit_ma$ designate the starting and ending addresses of matrix A data stored in memory, while $sw_addr_base_mb$ and $sw_addr_limit_mb$ represent the starting and ending addresses of matrix B data. These two pairs of registers define the address ranges for matrix A and matrix B, respectively. Furthermore, sw_tim, sw_tip, and sw_tin specify the number of rows in matrix A, the number of columns in matrix A (which also corresponds to the number of rows in matrix B), and the number of columns in matrix B, respectively. These registers are used to determine the loading process for the two input matrices and to generate the row and column indices for the output matrix.

VI. SpFFMM Tensor Engine

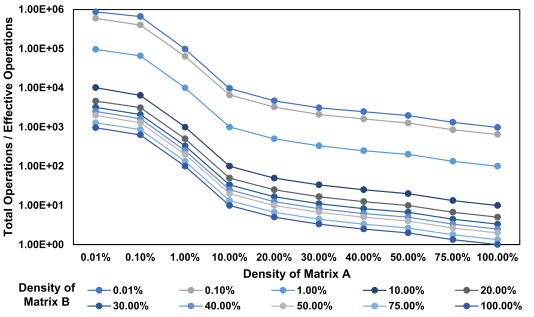
The sparse feature-feature MM (SpFFMM) is represented by the aggregation operator in both GCN and Vision GNN. To accommodate dynamic sparse MM, enhance PE utilization, and avoid redundant write operations, multi-SIMD processing arrays are employed as the core computational components. Conventional systolic arrays are not suitable for SpFFMM because sparse data must wait for another matrix element of the corresponding index to perform the computation at a specific timestamp, in accordance with the rhythm of another matrix input. This results in a significant number of clock cycles being wasted on waiting for the corresponding matrix element. These numerous idle clock cycles due to waiting can be eliminated if the sparse data is not paced according to the rhythm of another matrix input, but rather according to its own rhythm. Therefore, the SpFFMM TE utilizes a multi-SIMD processing array as its core computational component, with the sparse matrix side acting as the control side that loads another matrix element corresponding to the index.

The systolic array employed by the dense FFMM TE can fully reuse the row and column dimension data of MM to reduce memory accesses, thereby achieving lower access delay and energy consumption. For sparse matrices, data reuse can also be increased and memory accesses reduced through systolic data flow. This approach treats the sparse matrix as a dense matrix, and the systolic data setup feeds the current element into the systolic array regardless of whether it is a non-zero element or not, as illustrated in Fig. 10(a). Therefore, the computation time T for this sparse MM is the same as that for dense MM with the same dimensions expressed as below

$$T = \begin{cases} M + P, & if\ M > N \\ N + P, & if\ M < N \end{cases} \tag{9}$$

where M is the number of rows of matrix A in MM $A \times B = C$, P is the number of columns of matrix A or the number of rows of matrix B, and N is the number of columns of matrix B. From

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.



DATASET	Density of Adjacency Matrix (A)	Density of Matrix X (B)
Cora	0.14%	1.27%
Citeseer	0.08%	0.85%
Pubmed	0.02%	10.02%

Fig. 11. The computation reduction ratio of SpFFMM with sparsity-aware computation by systolic array.

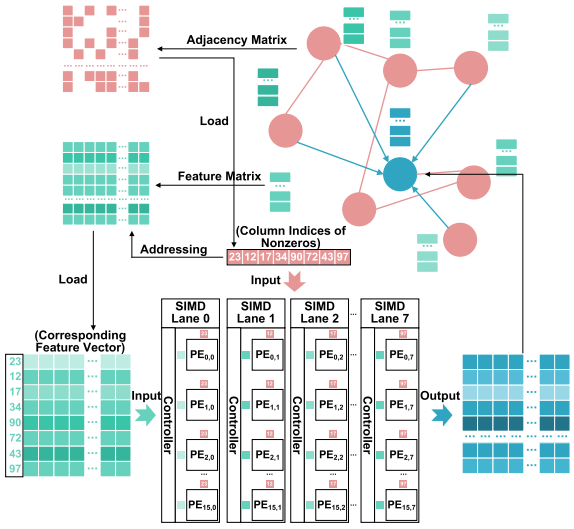


Fig. 12. Mapping of aggregation operation to SpFFMM tensor engine with multi-SIMD processing array.

this equation, we know that the utilization of this SpMM using systolic array is the density of the sparse matrix. If matrix A is 1024×1024 and matrix B is 1024×1024, the utilization of the systolic array is only 3.33% at 90% sparsity, which is unacceptable for AI acceleration tasks.

Another zero-skipping approach, illustrated in Fig. 10(b), involves removing columns 2, 4, and 6 from matrix A and the corresponding rows 2, 4, and 6 from matrix B. Since the indices of the corresponding elements in matrices A and B remain aligned, the correct result can be achieved. This method leverages matrix sparsity to eliminate redundant computations. The ratio of total computation to effective computation generated by this approach for various sparsity matrices is presented in Fig. 11. In the case of matrix A with a density of 0.10%, the redundant computation remains significantly high, leading to very low utilization of systolic arrays. Consequently, for extremely sparse MM, such as graph aggregation, systolic arrays are not deemed suitable.

The aforementioned analysis demonstrates that the systolic array is unsuitable as a computational component for SpFFMM. Systolic array follows a fixed data flow to compute matrix multiplication. However, sparse skip-zero computations require the array to flexibly select values to be involved in the computation, avoiding zero values. The PEs in the FFMM engine have strong data dependencies, meaning that even if a PE does not actually need a specific value for its computation, the data must still be passed through the entire array to satisfy the needs of other PEs. Due to these data dependencies, each PE in the array requires the same amount of processing time. If fine-grained skip-zero computations were to be implemented, the computation delays for different PEs could vary significantly, as non-zero values are typically randomly distributed. This would result in incorrect calculations across the entire array. As stated in Fig. 10, the FFMM engine's use of sparsity can only occur at a coarse granularity, which limits its ability to fully exploit sparsity when processing highly sparse computations. The data in Fig. 11 supports this observation. However, SIMD computations are highly flexible, with no data dependencies between different SIMD channels, and the PEs within the same channel only need to follow the same instructions, but there are no data dependencies. That is why SIMD allows for fine-grained skip-zero computations as shown in Fig. 12. Consequently, the multi-SIMD array is employed to execute SpFFMM. The flexibility of Multi-SIMD processing array enables the elimination of the aforementioned numerous idle clock cycles caused by waiting, as long as the sparse data is processed according to its own rhythm instead of being paced by another matrix input. SpFFMM is represented by the aggregation operation of GCN and Vision GNN, which is characterized by the fact that all input data are dynamic. The mapping of the computation onto the multi-SIMD processing array architecture is illustrated in Fig. 12. This architecture comprises 8 SIMD channels, each capable of processing 16 parallel data streams. Each SIMD channel is equipped with its own independent controller, enabling it to handle vector inner product operations of varying lengths.

VII. Non-Matrix Multiplication Engines

A. Sort Engines

Many neural network models require sorting algorithms. For example, the Top-K operator is commonly used to select the K most important data points. The Top-K operator is widely utilized, and both Transformers and GNNs depend on it. Top-K can be implemented based on sorting algorithms. There are a large number of data dependencies in sorting algorithms, making it difficult to parallelize them. From the 5th line of the pseudocode, it can be seen that the insertion sort algorithm has cross-iteration dependencies. Therefore, its loop is difficult to unroll, thus making it hard to achieve computational parallelization. By analyzing the number of iterations, the time complexity of this algorithm is $O(n^2)$, where n is the length of sequence A.

At the algorithmic level, common sorting algorithms include bubble sort, selection sort, insertion sort, quick sort, counting sort, bucket sort, radix sort and merge sort [30]. Their time and space complexities are shown in Table I. The first 5 algorithms are called comparison sorts. For any comparison sort, the time complexity in the worst case is at least $O(nlogn)$. The

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TIME AND SPACE COMPLEXITY OF COMMON SORTING ALGORITHMS

Sorting Algorithm	Average Time	Best Case	Worst Case	Space Complex.
Bubble Sort	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

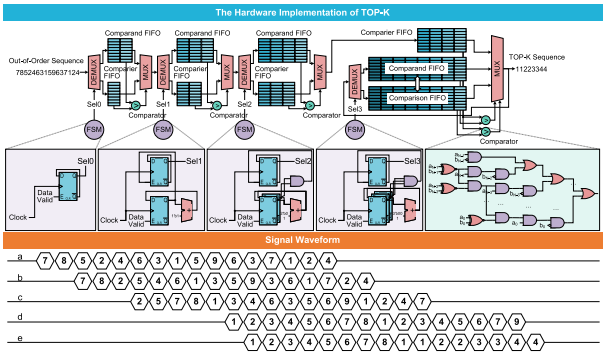


Fig. 13. The hardware implementation of merge sorting.

algorithms with the optimal time complexity are merge sort and heap sort. The last 3 algorithms are called linear sorts, and their time complexity has a linear relationship with the sequence length n and the data range k.

To minimize hardware resource overhead while maintaining computation time, the merge sorting algorithm is implemented in hardware circuits, with its architecture illustrated in Fig. 13. The merge sort hardware comprises demultiplexers, FIFOs, and comparators. Specifically, the demultiplexer is tasked with distributing input data to two FIFOs of varying depths; data from odd arrays is routed to a deep FIFO, while data from even arrays is directed to a shallow FIFO. The FIFOs serve as temporary storage, and at any given time, for an n-Merger, up to n+1 data elements are concurrently stored in the FIFOs. The comparator positioned after the FIFOs compares the first data element from each of the two FIFOs and determines the sequence for forwarding to the next level based on their values. By integrating 1-Merger, 2-Merger, 4-Merger, and so forth, the recursive process of merge sort is streamlined through pipelining and parallelization. Consequently, the sorting time of this circuit is reduced to $2n+\log_2 n-1$, with a corresponding depth of FIFOs utilized being $2n + \log_2 n - 2$.

To support TOP-K operations, one approach is to sort the input sequences using merge sort and then extract the largest or smallest data. However, this method results in redundant operations since, for TOP-K, only the first K data elements need to be compared, and further comparisons are unnecessary. Therefore, the architecture incorporates a TOP-K merger. This component selects the largest or smallest K data elements from multiple sets of sorted sequences of length K, thereby

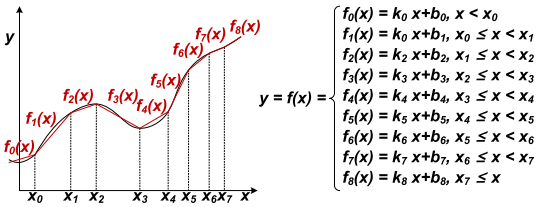


Fig. 14. Linear interpolation lookup table to implement nonlinear function.

eliminating redundant sorting of the data. The architecture is depicted in Fig. 13.

B. Nonlinear Engines

Existing computer algorithms implement nonlinear operations in three primary ways: the CORDIC algorithm, Taylor series approximation, and the use of lookup tables [31]. Specifically, the CORDIC algorithm transforms nonlinear operations into a series of addition, subtraction, and shift operations. Taylor series approximation, on the other hand, converts nonlinear operations into polynomial operations. Lastly, the lookup table method stores the results of nonlinear operations in a lookup table, allowing the operation results to be retrieved by querying the table. For nonlinear operators, there are notable differences in computational accuracy, speed, memory resource consumption, and overall computational resource consumption among the aforementioned methods. In general, the CORDIC algorithm, Taylor series approximation, and direct lookup table method can all achieve ultra-high computational accuracy. However, due to control dependencies, the CORDIC algorithm faces significant limitations in parallelized computation, thus hindering high-speed computation. Although Taylor series approximation can achieve high-speed computation through parallelization, it demands extensive computational resources due to the numerous multiplication operations involved. The direct lookup table method enables rapid computation but incurs substantial memory resource overhead to achieve high accuracy. In comparison to these three methods, the linear interpolation lookup table method, as illustrated in Fig. 14, offers a trade-off between memory and computational resource consumption, achieving a balance between fast computation speed and high computational accuracy [32].

Exponentiation (exp), reciprocal, and square root reciprocal (1/sqrt) are the fundamental nonlinear operations. These operations are linearly interpolated, and by combining them with arithmetic operations, we can compute nonlinear operators such as Softmax and GeLU. The data flow for these operators is as follows. For Softmax, the exponential value of each input x_i is first computed. These exponential values are then summed, and the reciprocal of the sum is taken. Finally, this reciprocal is multiplied by the exponential value of each x_i to obtain the Softmax value for each x_i. For GeLU, we simply configure the nonlinear function operator as a GeLU function.

To accommodate the data flow of the three nonlinear operators, the targeted computational architecture for nonlinear operators is illustrated in Fig. 15. This architecture comprises two nonlinear function calculators, one accumulator, two multipliers, one memory unit, three multiplexers,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

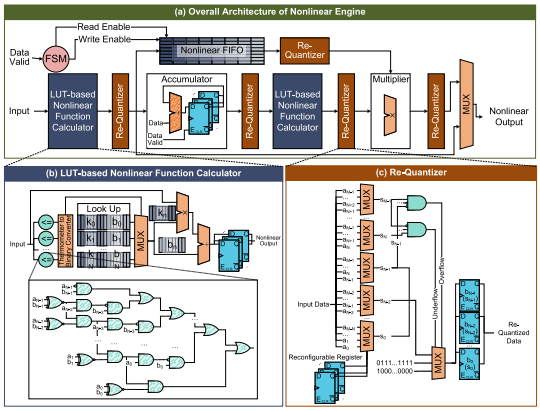


Fig. 15. The architecture of nonlinear engine: (a) Overall architecture, (b) LUT-based Nonlinear Function Calculator, (c) Re-Quantizer.

TABLE II
EVALUATION RESULTS

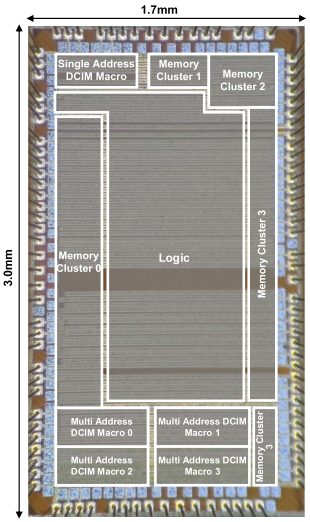
Model	Pyramid ViG-Ti	ViT-B/16	ResNet-50	GCN
Dataset	ImageNet	ImageNet	ImageNet	PubMed
Test Accuracy (%)	91.67 (TOP-5)	94.28 (TOP-5)	89.70 (TOP-5)	76.0
Accuracy Loss (%)	2.702	3.228	3.160	1.0
Latency (ms)	147.96	1374.57	334.54	3.13
Ener. Effi. (TOPS/W)	4.53	4.54	4.30	6.77

and five re-quantizers. The nonlinear function calculators are employed to execute nonlinear operations, the accumulator performs accumulation tasks, the multipliers handle multiplication operations, the memory unit temporarily stores computation results, and the multiplexers select either input data, the outcome of a nonlinear operation, or the result of a multiplication. The nonlinear function calculator consists of a comparator, a lookup table, a MAC unit, and four sets of registers, as shown in Fig. 15(b). The input x is first processed by the comparator, which determines the address of the lookup table that needs to be accessed based on the range of x. This address is then used to retrieve the slope k and intercept b of the linear function corresponding to the current value of x. Finally, the input x, along with the slope k and intercept b, are fed into the MAC unit to obtain the result of nonlinear function. Additionally, a re-quantizer, as shown in Fig. 15(c), is placed after each nonlinear function calculator, multiplier, accumulator, and memory unit to convert high bit-width results into low bit-width outcomes.

VIII. EXPERIMENTAL RESULTS

A. Chip Implementation and Measurement

The Quartet AI accelerator has been implemented and fabricated in 22-nm CMOS technology. Fig. 16 depicts the die photograph and a summary of the chip's specifications. The Quartet chip is featured with die size of 1.7mm ×3.0mm, on-chip SRAM buffer size of 506KB, and 384 normalized INT8 MAC units including both CIM and PEs. The Quartet chip has been evaluated across a wide range of AI models,



Technology	22 nm
Data Precision	INT8 (IN: INT8, W: INT8, OUT:32)
Die Size	1.7 mm × 3.0 mm
Number of MACs	384
On-Chip Buffer Size (KB)	506
Single Address DCIM	32 × 32 MACs @1b×8b
Multiple Address DCIM	4 × 8 × 16 MACs @1b×8b
Systolic Array	8 × 8 MACs @8b×8b
SIMD	8 × 16 MACs @8b×8b
Supply Voltage (V)	0.76~0.92V
Frequency (MHz)	75~152 (Overall) 260~401 (SpFWMM) 261~367 (FWMM) 105~154 (SpFFMM) 140~215 (FFMM)
Target	General (ViG, ViT, GNN, CNN, etc.)
Computation Engine	Digital CIM, Digital PEs
Peak Performance (TOPS)*1	0.16*2
Energy-Efficiency (TOPS/W)	4.52*3 4.30*4 (FWMM) 5.42*5 (FFMM) 3.72*6~12.30*7 (SpFWMM) 3.80*8~47.50*9 (SpFFMM)

*1: One operation represents one multiplication, addition or comparison.
*2: Measured at the highest performance point, 0.92V, 152MHz.
*3: Measured at the highest efficiency point, 0.8V, 121MHz.
*4: Measured at the highest efficiency point for FWMM, 0.76V, 261MHz.
*5: Measured at the highest efficiency point for FFMM, 0.80V, 156MHz.
*6: Measured at the highest efficiency point for SpFWMM, 0.76V, 260MHz.
*7: Low point: sparsity=0%. High point: Evaluated on PubMed of GCN Model.
*8: Measured at the highest efficiency point for SpFFMM, 0.76V, 105MHz.
*9: Low point: sparsity=0%. High point: sparsity=92%.

Fig. 16. The die photo and specifications of Quartet chip.

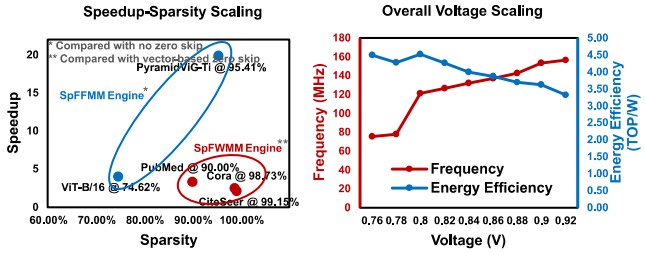


Fig. 17. Quartet's speedup-sparsity scaling and overall voltage scaling.

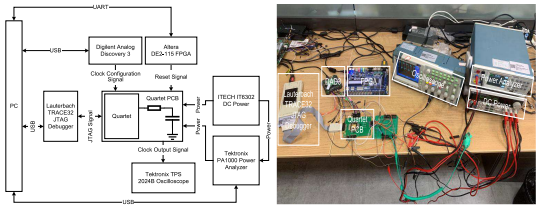


Fig. 18. The diagram and photo of test platform for Quartet.

with the results presented in Table II. Specifically, when tested on Pyramid ViG-Ti [11], ViT-B/16 [33], and ResNet-50 [6] using INT8 data format and ImageNet dataset, the achieved test accuracies are 91.67%, 94.28%, and 89.70%, respectively. For GCN model [2], the test accuracy is 76.0%, which is only 1.0% lower than that of the FP32 format. On the PubMed dataset, Quartet achieves a latency of only 3.13ms per GCN run, with energy efficiency of 6.77 TOPS/W. GCNs exhibit extremely high sparsity, and Quartet's performance benefits from a series of optimizations tailored for GCNs, including sparse processing based on the multi-address DCIM and the collaborative operation of heterogeneous cores. The overall energy efficiency of Quartet system is 4.52 TOPS/W when processing dense INT8 data, operating at 0.8V and 121MHz. The speedup-sparsity scaling and voltage scaling results of Quartet chip are illustrated in Fig. 17.

The test platform for Quartet is shown in Fig. 18, which contains the Quartet chip PCB, Digilent Analog Discovery

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

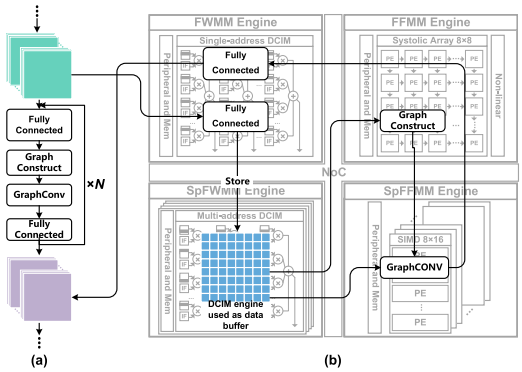


Fig. 19. (a) The structure of Vision GNN. (b) The mapping of Vision GNN onto Quartet.

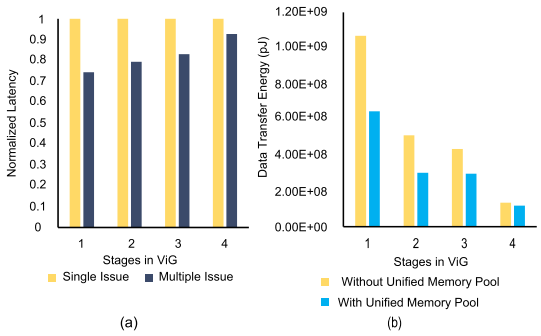


Fig. 20. Experimental results of improvements brought by (a) multiple-issue scheme and (b) unified memory pool.

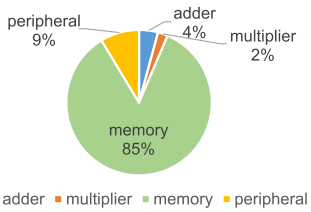


Fig. 21. Area breakdown of multi-addresss DCIM.

3 (DAD3) for generating clock configuration signals, Altera DE2-115 development board for generating reset signals, oscilloscope for observing waveforms, DC power for supplying power, power analyzer for measuring power consumption, and Lauterbach TRACE32 JTAG Debugger for data communication with the Quartet chip.

B. Quartet Dataflow: Using Vision GNN as an Example

In practical operation, operators with different characteristics of AI workloads are deployed to the appropriate engine for their respective computations. The data flow between operators is managed through the NOC between engines. Taking Vision GNN as an example, Vision GNN is a typical graph fusion algorithm as Fig. 19. Its main operators include fully connected layers, graph construction, and graph convolution, which correspond to FWMM, FFMM, and SpFFMM, respectively. A single iteration of the core structure involves the sequential execution of fully connected, graph construction, graph convolution, and fully connected operations, with each inference consisting of N iterations. For the fully connected operator, the input feature map undergoes rearrangement, tiling, padding, and other steps before being fed into the FWMM tensor engine, where it performs matrix multiplication with the weights stored in the DCIM array. The output feature map of the fully connected operator is stored in the SpFWMM tensor engine for use by the subsequent graph construction operator. In the FFMM tensor engine, the graph construction operator performs FFMM on the loaded features, sorts them, and produces the graph adjacency matrix, which is sent to the SpFFMM tensor engine. Upon receiving the graph adjacency matrix, the SpFFMM tensor engine begins the graph convolution computation, loading the corresponding feature vectors from the SpFWMM tensor engine based on the column indices of the non-zero elements in the adjacency matrix. The graph convolution results are then sent to the FWMM tensor engine for the fully connected operation, yielding the output feature map for that iteration.

C. DCIM Result

The multi-address DCIM macros are designed based on customized development process using Cadence Virtuoso. To clearly demonstrate the improvements brought by customized development, we further used Verilog to develop a latch-based DCIM macro with the same memory size. The latch-based DCIM macro was synthesized using Cadence Genus to obtain the area. The area of the multi-address DCIM macro based on the customized design is approximately 30% of that of the DCIM developed and integrated based on Verilog, which demonstrates the improvement brought by customized design.

To analyze the overhead of multi-address, we have analyzed area breakdown of multi-address DCIM according to the layout. The area of the peripheral circuit accounts for only 9% of the total area. This indicates that, even with the additional overhead caused by multi-address, the area ratio of the peripheral circuit remains relatively small.

During the design of our system, in order to simplify the complexity of the physical design, we did not allocate a separate voltage domain for DCIM. This design choice made it difficult to obtain the separate power consumption of DCIM.

D. System Result Analysis and Comparisons

The multi-issue allows multiple operators or tiles in a computation task to be executed in parallel, which reduces the overall computational latency. The reduction in computational latency for each stage of Vision GNN is shown in Fig. 21(a). For Stage 1, Stage 2, Stage 3, and Stage 4 of Vision GNN, the computational latency is reduced by 25.58%, 20.54%, 16.95%, and 7.24%, respectively.

The implementation of on-chip unified memory pool in Quartet leads to a reduction in both off-chip and on-chip accesses, thereby enhancing the overall energy efficiency. Fig. 21(b) illustrates the data transfer energy overhead for each stage of Vision GNN, both with and without the on-chip unified memory pool. Specifically, for Stage 1, Stage 2,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON WITH STATE-OF-THE-ART WORKS

	ISSCC'23 [34]	JSSC'24 [35]	ISSCC'23 [36]	JSSC'23 [27]	ISSCC'23 [28]	This Work
Technology	28nm	28nm	28nm	28nm	28nm	22nm
Voltage (V)	0.66∼1.3	0.46∼1.13	0.64∼1.03	0.6∼1.0	0.6∼1.0	0.76∼0.92
Frequency (MHz)	100∼500	20∼400	20∼320	50∼220	115∼290	75∼152
Target	CNN	CNN, ViT	Transformer	CNN, GNN	GNN	General (ViG, CNN, ViT, GNN, etc.)
Data Precision	INT8	INT1∼INT8	INT8	INT8/INT16 /BF16/FP32	INT8/INT16 /FP32	INT8
Computation Engine	Digital PEs	Digital PEs	Digital CIM	Digital CIM	Digital CIM	Digital CIM + PEs
On-Chip Buffer (KB)	1120	200	240	140	640	506
Die Area (mm^2)	9.08	9.61	3.93	6.69	12.42	5.10
Power (mW)	17∼174	2.06∼237.1	8.27∼250.65	12.5∼69.4	29.1∼166.8	17.13∼48.27
Overall Energy Efficiency (TOPS/W)	7.5∼11.2	8.94∼127.8	1.96∼25.22	1.76mJ/inference (FP32, GCN)	0.312mJ/inference (FP32, GCN)	0.09mJ/inference (GCN) 4.52 (Dense Data)
FWMM TOPS/W	7.5∼11.2	8.94 (Sparsity=10%)	1.76	36.5(INT8) 3.7(FP32)	N/A	4.30 (Dense Data)
FFMM TOPS/W	N/A	N/A	3.41	N/A	N/A	5.42 (Dense Data)
SpFWMM TOPS/W	N/A	127.8 (Sparsity=90%)	N/A	N/A	13.1∼85.0 (INT8) 1.3∼8.3 (FP32)	3.72∼12.30
SpFFMM TOPS/W	N/A	N/A	53.83	35.1nJ/gather (FP32, GCN)	3.7nJ/gather (FP32, GCN)	0.30nJ/gather (GCN) 3.80∼47.5

Stage 3, and Stage 4 of Vision GNN, the data transfer energy overhead exhibits a reduction of 39.51%, 40.95%, 31.73%, and 12.46%, respectively.

Quartet is compared with state-of-the-art related works, as presented in Table III. For the GCN computation on PubMed, Quartet demonstrates an energy efficiency of 0.09 mJ/inference, which is 19.56× and 3.47× higher than ReDCIM [26], [27] and TensorCIM [28], respectively. When it comes to the SpFFMM tensor engine, Quartet achieves an impressive energy efficiency of 0.30 nJ/Gather, outperforming ReDCIM and TensorCIM by 117.00× and 12.33×, respectively. Even when normalized to INT8, Quartet's advantage remains evident. Furthermore, for the computationally intensive operations of FWMM and FFMM, Quartet's energy efficiency surpasses that of ISSCC 2023 16.2 [36] by 2.44× and 1.59×, respectively. In comparison to [34] and [35], which are limited to a few specific models, Quartet offers the versatility to support a wider range of AI algorithms.

E. Discussion

By dividing the various AI workloads into four categories based on their computing characteristics and designing optimized computing engines specifically for each category, Quartet has achieved the goal of providing high energy efficiency computing power while maintaining high versatility. But just like most heterogeneous systems, Quartet also faces some challenges, such as utilization. While each engine of Quartet can efficiently handle specific workloads, when combined into the heterogeneous system Quartet, it becomes challenging to ensure optimal utilization of each engine during task execution. This can lead to idle computational resources and a reduction in overall system performance.

Another important aspect is scalability. The Quartet benefits from its modular design and has excellent scalability. Quartet is currently integrated with four heterogeneous engines connected through NoC, making it scalable by increasing the number of engines or enhancing the computational power within each individual engine. For handling larger matrix computations, there are two methods. One approach is tiling the matrix computation, which divides a large matrix calculation into multiple smaller matrix computations, allowing large matrix handling without the need for expanding chip computational power or storage capacity. The other method involves increasing the number of engines or adding more PEs or DCIM macros within a single engine.

To further enhance the energy efficiency of Quartet in the future, improving the utilization of heterogeneous systems and adopting other methods such as software-hardware co-design approximate computing [37] are all possible directions. For utilization, one potential approach is to simplify the existing four engines into two: DCIM-based and SIMD-based engines, handling FFMM and FWMM, respectively. Both engines would be reconfigurable arrays that, under peripheral logic control, could be configured for dense or sparse computation modes, thereby covering all four target workloads. This would help alleviate the utilization issue to some extent, making Quartet more adaptable to other AI models. The hardware-software co-design approximate computing is a highly promising research direction which is orthogonal to our current architecture-level optimizations, and it could potentially be integrated into future enhancements of our digital DCIM designs to help improve the system efficiency.

Furthermore, Quartet currently relies on manual compilation. When the workload is substantial, manual compilation can result in a heavy workload for human, and it is difficult to ensure both correctness and efficiency. Therefore, developing a dedicated compiler adapted for this system could be a potential direction moving forward.

IX. CONCLUSION

This paper investigates a generalized hardware accelerator Quartet for various AI algorithms with

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.

sparse-dense-static-dynamic heterogeneous tensor engines. These four TEs are equipped with single-address SRAM digital CIMs, multi-address SRAM digital CIMs, systolic array, and multi-SIMD array, to efficiently handle different matrix multiplications, e.g., FWMM, SpFWMM, FFMM, and SpFFMM, respectively. The sort and non-linear operators are implemented by merge sort and linear interpolation table lookup methods, respectively. In addition, this paper proposes an operator-block dual-level multi-issue mechanism to achieve inter-operator and inter-block dual-level parallelization. To reduce the dependence on off-chip memory, this paper constructs an on-chip unified memory pool to enhance the on-chip data flow. Consequently, the proposed Quartet chip achieves leading energy efficiency and versatility to support transformers, GCNs, ViG, CNNs, and so on. Specifically, for GCN on PubMed, it attains 19.56× and 3.47× energy efficiency improvement compared with related works ReDCIM and TensorCIM, respectively.

REFERENCES

[1] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 30, 2017, pp. 5998–6008.

[2] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2016.

[3] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[4] E. Akleman, "Deep learning," *Computer*, vol. 53, no. 9, pp. 436–444, Sep. 2020.

[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent.*, 2015.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.

[8] Q. Dong et al., "15.3 a 351TOPS/W and 372.4GOPS compute-in-memory SRAM macro in 7 nm FinFET CMOS for machine-learning applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 242–244.

[9] Y.-D. Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[10] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[11] K. Han, Y. Wang, J. Guo, Y. Tang, and E. Wu, "Vision GNN: An image is worth graph of nodes," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, 2022.

[12] E. Qin et al., "SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 58–70.

[13] M. Yan et al., "HyGCN: A GCN accelerator with hybrid architecture," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 15–29.

[14] T. Geng et al., "AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2020, pp. 922–936.

[15] J. Li, A. Louri, A. Karanth, and R. Bunescu, "GCNAX: A flexible and energy-efficient accelerator for graph convolutional neural networks," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, Feb. 2021, pp. 775–788.

[16] T. Geng et al., "I-GCN: A graph convolutional network accelerator with runtime locality enhancement through islandization," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchit.*, Oct. 2021, pp. 1051–1063.

[17] R. Xie, J. Yin, and J. Han, "DyGA: A hardware-efficient accelerator with traffic-aware dynamic scheduling for graph convolutional networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 12, pp. 5095–5107, Dec. 2021.

[18] K. Kiningham, P. Levis, and C. Ré, "GRIP: A graph neural network accelerator architecture," *IEEE Trans. Comput.*, vol. 72, no. 4, pp. 914–925, Apr. 2023.

[19] A. I. Arka, J. R. Doppa, P. P. Pande, B. K. Joardar, and K. Chakrabarty, "ReGraphX: NoC-enabled 3D heterogeneous ReRAM architecture for training graph neural networks," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1667–1672.

[20] W.-C. Huang et al., "A 28-nm 25.1 TOPS/W sparsity-aware CNN-GCN deep learning SoC for mobile augmented reality," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 42–43.

[21] T. Yang et al., "PIMGCN: A ReRAM-based PIM design for graph convolutional network acceleration," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 583–588.

[22] Y. He, Y. Wang, C. Liu, H. Li, and X. Li, "TARe: Task-adaptive in-situ ReRAM computing for graph learning," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 577–582.

[23] N. Challapalle, K. Swaminathan, N. Chandramoorthy, and V. Narayanan, "Crossbar based processing in memory accelerator architecture for graph convolutional networks," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2021, pp. 1–9.

[24] Z. Zhang et al., "A 28nm 16kb aggregation and combination computing-in-memory macro with dual-level sparsity modulation and sparse-tracking ADCs for GCNs," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.

[25] Y. Wang, M. Yang, S. Xie, M. Wang, and J. P. Kulkarni, "CIMGN: An energy-efficient all-digital compute-in-memory graph neural network processor," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 477–480.

[26] F. Tu et al., "A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[27] F. Tu et al., "ReDCIM: Reconfigurable digital computing- in -memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.

[28] Y. Wang et al., "TensorCIM: Digital computing-in-memory tensor processor with multichip-module-based architecture for beyond-NN acceleration," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 734–747, Feb. 2025.

[29] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*. Reading, MA, USA: Addison-Wesley, 2010.

[30] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction To Algorithms*, 3rd ed., Cambridge, MA, USA: MIT Press, 2009.

[31] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*. London, U.K.: Oxford Univ. Press, 2009.

[32] J. Yu et al., "NN-LUT: Neural approximation of non-linear operations for efficient transformer inference," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 577–582.

[33] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent.*, 2021.

[34] C.-Y. Du et al., "A 28nm 11.2TOPS/W hardware-utilization-aware neural-network accelerator with dynamic dataflow," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.

[35] S. Moon, H.-G. Mun, H. Son, and J.-Y. Sim, "Multipurpose deep-learning accelerator for arbitrary quantization with reduction of storage, logic, and latency waste," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 143–156, Jan. 2024.

[36] S. Liu et al., "16.2 a 28 nm 53.8TOPS/W 8b sparse transformer accelerator with in-memory butterfly zero skipper for unstructured-pruned NN and CIM-based local-attention-reusable engine," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 250–252.

[37] B. Liu et al., "Layer-wise mixed-modes CNN processing architecture with double-stationary dataflow and dimension-reshape strategy," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 10, pp. 4652–4664, Oct. 2024.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.



Yikan Qiu received the B.E. degree in microelectronic science and technology from Nanjing University of Science and Technology, Nanjing, China, in 2019, and the Ph.D. degree from the School of Integrated Circuits, Peking University, Beijing, China, in 2024. He is currently a Senior Logic Design Engineer with Hangzhou Hikvision Digital Technology Company Ltd. His research interests include graph computing, deep learning acceleration, and domain-specific architecture.



Guoxiang Li received the B.S. degree in electronic and information science and technology from the School of Electronics Engineering and Computer Science, Peking University, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits. His research interests include model compression, deep learning accelerators, and software–hardware co-design for neural networks.



Meng Wu (Graduate Student Member, IEEE) received the B.S. degree from the Institute of Microelectronics in 2020 and the Ph.D. degree from the School of Integrated Circuits, Peking University, Beijing, China, in 2025. He is currently a Custom IP Design Engineer with Bytedance. His research interests include in-memory and near-memory computing architecture, embedded memory design, custom circuit design, and mixed-signal processing.



Yifan Jia received the B.S. degree in integrated circuit design and integrated systems from the School of Electronics Engineering and Computer Science, Peking University, Beijing, China, in 2024. He is currently pursuing the Ph.D. degree in integrated circuit design with the School of Integrated Circuits, Peking University. His research interests include AI algorithm compression, model lightweighting, AI accelerator design, and hardware–software co-design.



Le Ye (Senior Member, IEEE) was born in Jiangsu, China, in 1983. He received the B.S. degree (Hons.) in physics from the Department of Physics, Nanjing University, Nanjing, China, in 2005, and the Ph.D. degree in microelectronics from the Institute of Microelectronics, Peking University, Beijing, China, in 2010.

From April 2009 to April 2010, he was a Visiting Scholar at the Institute of Circuits and Systems, Dresden University of Technology, Dresden, Germany. Since 2010, he has been a Faculty Member with Peking University, Hangzhou, China, where he is currently an Associate Professor. He is also the Dean of the IoT Research Center, Advanced Institute of Information Technology, Peking University. He has authored or co-authored more than 40 articles. He is the holder of over 20 granted patents. His research interests include analog and mixed-signal IC design, ultra-low power IC design, computing-in-memory AI chip design, and implanted IC and micro-system for medical applications. He was a recipient of the National Science Fund for Outstanding Young Scholars and Beijing New-Star Plan of Science and Technology in China. He has been an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS since 2020.



Yufei Ma (Senior Member, IEEE) received the B.S. degree in information engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2011, the M.S.E. degree in electrical engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2013, and the Ph.D. degree from Arizona State University, Tempe, AZ, USA, in 2018. From 2018 to 2019, he was with FABU America Inc., Tempe, where he worked on high-performance ASIC chips for ADAS/AD applications. From 2019 to 2020, he was with Nanjing University, Nanjing, as a Research Associate Professor. In 2020, he joined the Institute for Artificial Intelligence and the School of Integrated Circuits, Peking University, Beijing, China, as an Assistant Professor. His current research interests include energy-efficient AI/ML accelerators, in-memory computing, neuromorphic computing, and algorithm-hardware co-design for deep learning. In the field of integrated circuits and hardware architecture, over 20 articles as a first author or a corresponding author have been published. He has served as a TPC Member for the ACM/IEEE Design Automation Conference (DAC).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:20 UTC from IEEE Xplore. Restrictions apply.